Module 5 Activity 2: Using Map Algebra

Purpose:

Incorporate the ***arcpy.sa*** module to work with map algebra operators to generate new raster datasets to determine high avalanche risk based on the parameters listed below.

Instructions:

Any Python environment will work for this exercise.

Parameters:

When conditions are right, avalanches tend to occur consistently based on the following criteria:

- **Elevation**: between 1,900m and 2,700m
- **Slope**: between 30 degrees and 45 degrees
- **Aspect**: south

NOTE: the above criteria are very broad and generalized. Many other factors are involved for an avalanche to occur, such as temperature, wind, snowpack and many others. This activity only provides an example on how to determine suitable locations.

Download the data Module 5 Activity 2 and save the data to your working directory.

1. Import the ***arcpy*** module
2. Import the ***arcpy.sa*** module
3. Check out the spatial analyst extension so it can be used
4. Add a ***print*** statement to the ***CheckOutExtension*** function to notify the user that it is ready for use
5. Set the workspace and overwrite outputs
6. Define geoprocessing messages to be used throughout the script. Use the messages for all geoprocessing tools
7. Use a list to describe the DEM and retrieve the following:
    - Raster name
    - Cell size
    - Spatial reference

Python

```
#import arcpy and spatial analyst modules
import arcpy
from arcpy import env
from arcpy.sa import *
#check out the spatial analyst extension
arcpy.CheckOutExtension("Spatial")
print ("Spatial Analyst extension checked out and ready to use.")
print ("") #blank print for ease of reading
#set the overwrite outputs and workspace
env.overwriteOutput = True
env.workspace = "C:\GEOS456\Avalanche.gdb"
#define GetMessages to print geoprocessing tool progress
def messages():
    print (arcpy.GetMessage(0))
    count = (arcpy.GetMessageCount())
    print (arcpy.GetMessage(count-1))
    print ("")
#get a list of the rasters in the geodatabase
rasterlist = arcpy.ListRasters()
#start a for loop to iterate through the list and describe raster properties
for rasters in rasterlist:
    desc = arcpy.Describe(rasters)
    print (rasters)
    print (desc.meanCellWidth)
    print (desc.spatialReference.name)
    print ("")
```

You can run the script at this point. This will print the results of the ***describe*** function. It may be useful to check what the cell size and coordinate system is before proceeding. This becomes especially important when working with grid statistics and overlaying rasters, which will be covered in the next objective.

The next steps will walk you through on how to create various surfaces from the DEM, including ***Slope*** and ***Aspect***

8. Create a variable called ***dem***
9. Use the ***arcpy.Raster*** function to define the raster you want to work with. This will need to be done to work with the raster calculator. In this case, we will use the raster from the ***list*** created in ***Step 6***

Python

```
#import arcpy and spatial analyst modules
import arcpy
from arcpy import env
from arcpy.sa import *
#check out the spatial analyst extension
arcpy.CheckOutExtension("Spatial")
print ("Spatial Analyst extension checked out and ready to use.")
print ("") #blank print for ease of reading
#set the overwrite outputs and workspace
env.overwriteOutput = True
env.workspace = "C:\GEOS456\Avalanche.gdb"
#define GetMessages to print geoprocessing tool progress
def messages():
    print (arcpy.GetMessage(0))
    count = (arcpy.GetMessageCount())
    print (arcpy.GetMessage(count-1))
    print ("")
#get a list of the rasters in the geodatabase
rasterlist = arcpy.ListRasters()
#start a for loop to iterate through the list and describe raster properties
for rasters in rasterlist:
    desc = arcpy.Describe(rasters)
    print (rasters)
    print (desc.meanCellWidth)
    print (desc.spatialReference.name)
    print ("")
#create a slope and aspect raster from the DEM and save them to the gdb
dem = arcpy.Raster(rasters) #dem must be defined to work with the Raster Calculator
```

10. Create a variable called ***slopeRaster***
11. Use the ***Slope*** tool to generate a slope surface from the ***dem***
12. Save the ***slopeRaster*** to the geodatabase called ***LL_Slope*** (Lake Louise Slope)
13. Create a variable called ***aspectRaster***
14. Use the ***Aspect*** tool to generate an aspect surface from the ***dem***
15. Save the ***aspectRaster*** to the geodatabase called ***LL_Aspect*** (Lake Louise Aspect)
16. Use the ***print*** statement to inform the user that the surfaces have been successfully created

Python

```
#import arcpy and spatial analyst modules
import arcpy
from arcpy import env
from arcpy.sa import *
#check out the spatial analyst extension
arcpy.CheckOutExtension("Spatial")
print ("Spatial Analyst extension checked out and ready to use.")
print ("") #blank print for ease of reading
#set the overwrite outputs and workspace
env.overwriteOutput = True
env.workspace = "C:\GEOS456\Avalanche.gdb"
#define GetMessages to print geoprocessing tool progress
def messages():
    print (arcpy.GetMessage(0))
    count = (arcpy.GetMessageCount())
    print (arcpy.GetMessage(count-1))
    print ("")
#get a list of the rasters in the geodatabase
rasterlist = arcpy.ListRasters()
#start a for loop to iterate through the list and describe raster properties
for rasters in rasterlist:
    desc = arcpy.Describe(rasters)
    print (rasters)
    print (desc.meanCellWidth)
    print (desc.spatialReference.name)
    print ("")
#create a slope and aspect raster from the DEM and save them to the gdb
dem = arcpy.Raster(rasters) #dem must be defined to work with the Raster Calculator
#create a slope raster
slopeRaster = Slope(dem)
print ("Processing slope...")
messages()
#save the slope raster
slopeRaster.save("C:\GEOS456\Avalanche.gdb\LL_Slope")
print ("Saving slope...")
messages()
#create an aspect raster
aspectRaster = Aspect(dem)
print ("Processing Aspect...")
messages()
#save the aspect raster
aspectRaster.save("C:\GEOS456\Avalanche.gdb\LL_Aspect")
print ("Saving Aspect...")
messages()
```

The next steps will use the ***map algebra*** to generate new raster datasets that satisfy the avalanche parameters outlined at the beginning of the activity

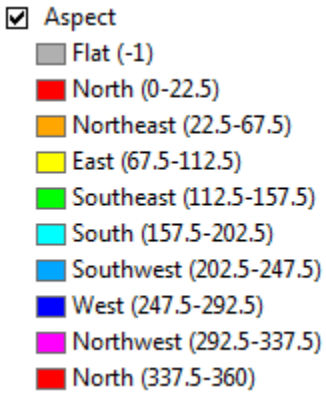
17. Create a variable called ***Avi_Elev*** (Avalanche Elevation)
18. Refer to the elevation criteria (**Elevation**: between 1,900m and 2,700m)
19. Save the elevation criteria raster to the geodatabase with the name ***Elev_Criteria***
20. Use the ***print*** statement informing the user that the raster has been successfully created
21. Repeat ***steps 16-19*** for slope and aspect ensuring you create a unique variable for each

Python

```
#import arcpy and spatial analyst modules
import arcpy
from arcpy import env
from arcpy.sa import *
#check out the spatial analyst extension
arcpy.CheckOutExtension("Spatial")
print ("Spatial Analyst extension checked out and ready to use.")
print ("") #blank print for ease of reading
#set the overwrite outputs and workspace
env.overwriteOutput = True
env.workspace = "C:\GEOS456\Avalanche.gdb"
#define GetMessages to print geoprocessing tool progress
def messages():
    print (arcpy.GetMessage(0))
    count = (arcpy.GetMessageCount())
    print (arcpy.GetMessage(count-1))
    print ("")
#get a list of the rasters in the geodatabase
rasterlist = arcpy.ListRasters()
#start a for loop to iterate through the list and describe raster properties
for rasters in rasterlist:
    desc = arcpy.Describe(rasters)
    print (rasters)
    print (desc.meanCellWidth)
    print (desc.spatialReference.name)
    print ("")
#create a slope and aspect raster from the DEM and save them to the gdb
dem = arcpy.Raster(rasters) #dem must be defined to work with the Raster Calculator
#create a slope raster
slopeRaster = Slope(dem)
print ("Processing slope...")
messages()
#save the slope raster
slopeRaster.save("C:\GEOS456\Avalanche.gdb\LL_Slope")
print ("Saving slope...")
messages()
#create an aspect raster
aspectRaster = Aspect(dem)
print ("Processing Aspect...")
messages()
#save the aspect raster
aspectRaster.save("C:\GEOS456\Avalanche.gdb\LL_Aspect")
print ("Saving Aspect...")
messages()
#use map algebra to create expressions to satisfy avalanche criteria
Avi_Elev = ((dem >= 1900) & (dem <= 2700))
#save the elevation criteria raster
Avi_Elev.save("C:\GEOS456\Avalanche.gdb\Elev_Criteria")
print ("Saving Elevation Criteria...")
messages()
#specify the slope criteria
Avi_Slope = ((slopeRaster >= 30) & (slopeRaster <= 45))
#save the slope criteria raster
Avi_Slope.save("C:\GEOS456\Avalanche.gdb\Slope_Criteria")
print ("Saving Slope criteria...")
messages()
#specify the aspect criteria
Avi_Aspect = ((aspectRaster >= 157.5) & (aspectRaster <= 202.5))
#save the aspect criteria raster
Avi_Aspect.save("C:\GEOS456\Avalanche.gdb\Aspect_Criteria")
print ("Saving Aspect criteria...")
messages()
```

<u>NOTE</u>: you must use actual values for the aspect expression. An expression stating "South" will not work as the Aspect tool does not understand what that value means. It isn't immediately evident what the South aspect values are. These values can be found

by research or by manually running the aspect tool in ArcMap. When running the aspect tool in ArcMap, the resulting surface displayed will break down the aspect in terms of degrees shown below.



Now that we have created a raster for each of the criteria, we want to combine all 3 rasters together to show where the highest likelihood of an avalanche can take place based on the parameters. This can be done in two ways:

- First is to ***ADD*** all the rasters together using the **+** operator
- Second is to ***MULTIPLY*** all the rasters together using the ***** operator

We will use both to compare the results.

22. Create a variable called ***FinalRaster1*** and add all the rasters together using an addition expression
23. Save the raster to the geodatabase called ***Final_Criteria_Add***
24. Use the ***print*** statement to inform the user that the raster has been successfully created
25. Create a variable called ***FinalRaster2*** and multiply all the rasters together using an multiplication expression
26. Save the raster to the geodatabase called ***Final_Criteria_Mult***
27. Use the ***print*** statement to inform the user that the raster has been successfully created
28. Return the spatial analyst extension back to the license manager

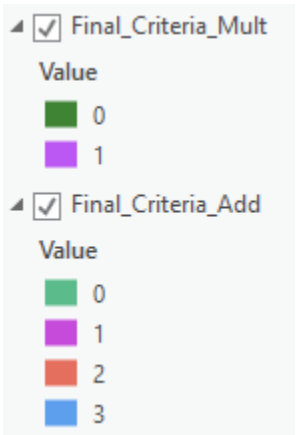
Python

```
#start a for loop to iterate through the list and describe raster properties
for rasters in rasterlist:
    desc = arcpy.Describe(rasters)
    print (rasters)
    print (desc.meanCellWidth)
    print (desc.spatialReference.name)
    print ("")
#create a slope and aspect raster from the DEM and save them to the gdb
dem = arcpy.Raster(rasters) #dem must be defined to work with the Raster Calculator
#create a slope raster
slopeRaster = Slope(dem)
print ("Processing slope...")
messages()
#save the slope raster
slopeRaster.save("C:\GEOS456\Avalanche.gdb\LL_Slope")
print ("Saving slope...")
messages()
#create an aspect raster
aspectRaster = Aspect(dem)
print ("Processing Aspect...")
messages()
#save the aspect raster
aspectRaster.save("C:\GEOS456\Avalanche.gdb\LL_Aspect")
print ("Saving Aspect...")
messages()
#use map algebra to create expressions to satisfy avalanche criteria
Avi_Elev = ((dem >= 1900) & (dem <= 2700))
#save the elevation criteria raster
Avi_Elev.save("C:\GEOS456\Avalanche.gdb\Elev_Criteria")
print ("Saving Elevation Criteria...")
messages()
#specify the slope criteria
Avi_Slope = ((slopeRaster >= 30) & (slopeRaster <= 45))
#save the slope criteria raster
Avi_Slope.save("C:\GEOS456\Avalanche.gdb\Slope_Criteria")
print ("Saving Slope criteria...")
messages()
#specify the aspect criteria
Avi_Aspect = ((aspectRaster >= 157.5) & (aspectRaster <= 202.5))
#save the aspect criteria raster
Avi_Aspect.save("C:\GEOS456\Avalanche.gdb\Aspect_Criteria")
print ("Saving Aspect criteria...")
messages()
#add all criteria rasters into one final avalanche suitability raster
finalRaster1 = ((Avi_Elev) + (Avi_Slope) + (Avi_Aspect))
#save the final raster
finalRaster1.save("C:\GEOS456\Avalanche.gdb\Final_Criteria_Add")
print ("Saving Final Raster (Addition)...")
messages()
#multiply all criteria rasters into one final avalanche suitability raster
finalRaster2 = ((Avi_Elev) * (Avi_Slope) * (Avi_Aspect))
#save the final raster
finalRaster2.save("C:\GEOS456\Avalanche.gdb\Final_Criteria_Mult")
print ("Saving Final Raster (Multiply)...")
messages()
#check in the spatial analyst extension
arcpy.CheckInExtension("Spatial")
print ("Spatial Analyst extension has been returned to the license manager.")
```

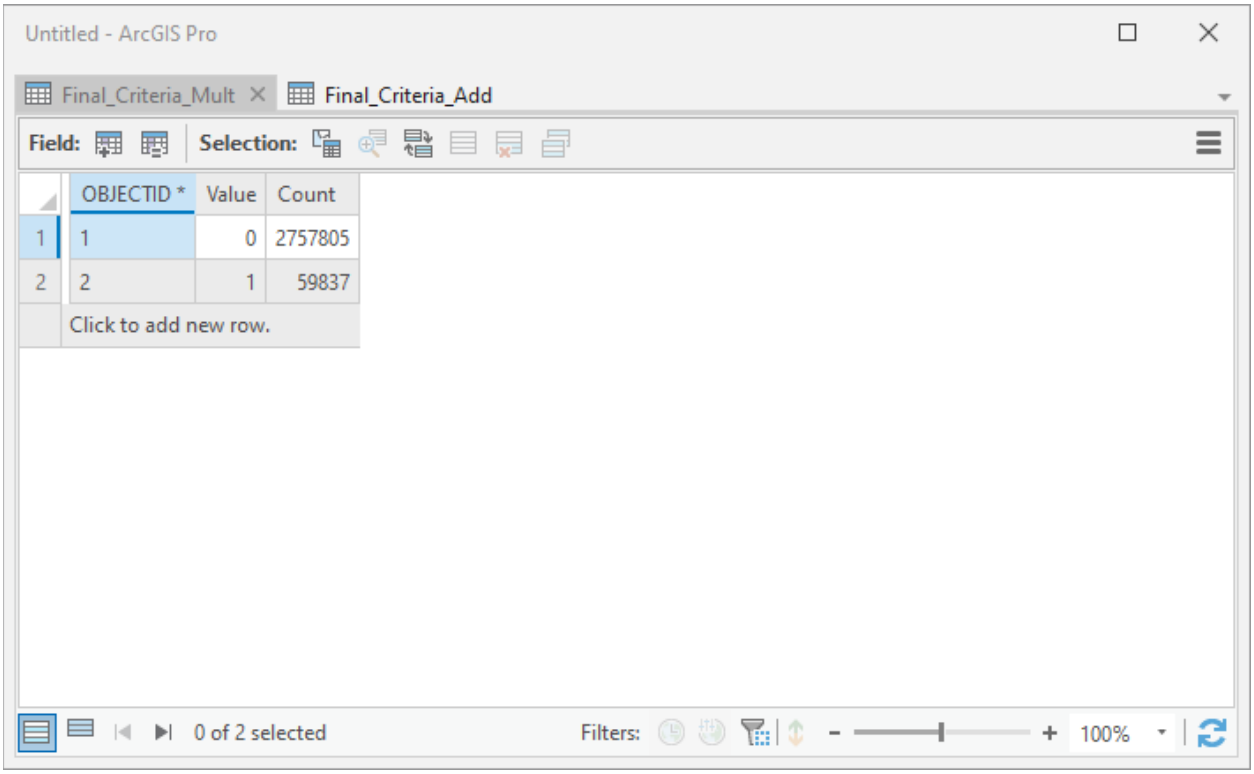
29. Run the script
30. Examine your results

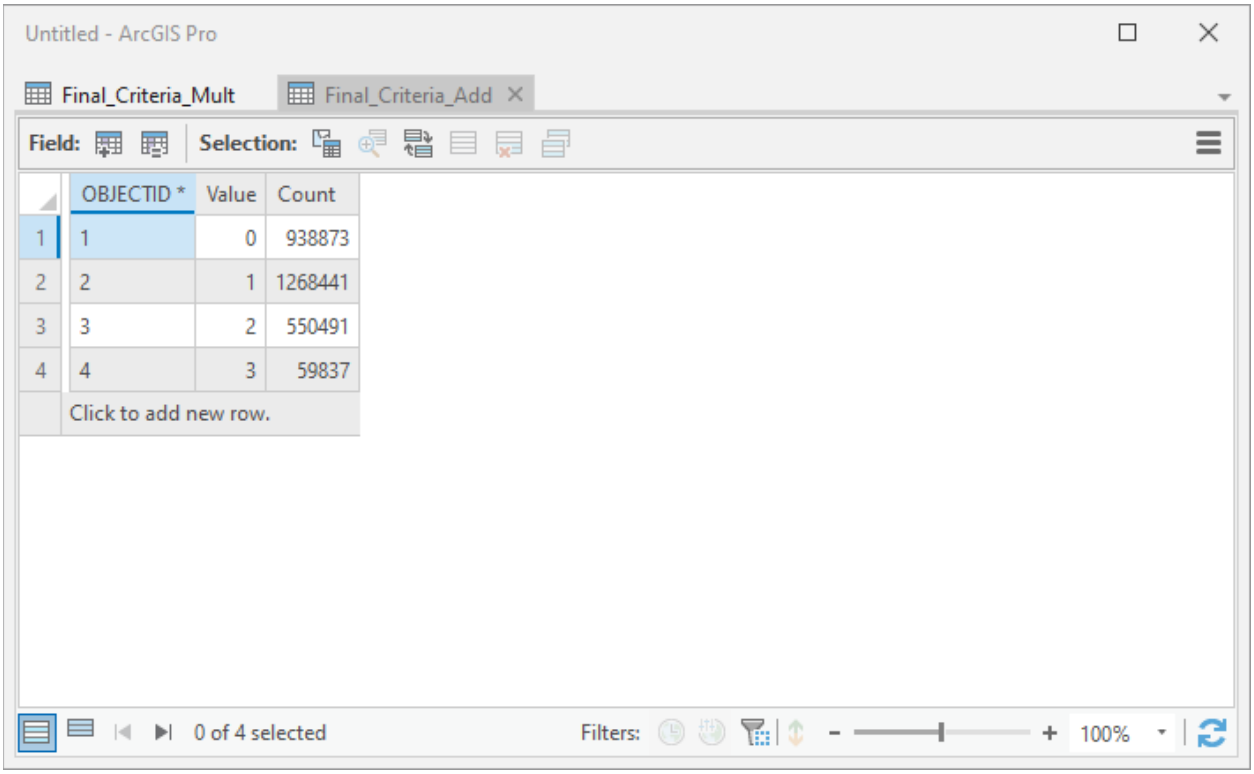
```
Python
Spatial Analyst extension checked out and ready to use.

LakeLouise_DEM
18.52099265575135
NAD_1983_UTM_Zone_11N

Processing slope...
Start Time: Saturday, August 21, 2021 11:23:07 AM
Succeeded at Saturday, August 21, 2021 11:23:07 AM (Elapsed Time: 0.00 seconds)

Saving slope...
Start Time: Saturday, August 21, 2021 11:23:07 AM
Succeeded at Saturday, August 21, 2021 11:23:07 AM (Elapsed Time: 0.00 seconds)

Processing Aspect...
Start Time: Saturday, August 21, 2021 11:23:09 AM
Succeeded at Saturday, August 21, 2021 11:23:12 AM (Elapsed Time: 2.27 seconds)

Saving Aspect...
Start Time: Saturday, August 21, 2021 11:23:09 AM
Succeeded at Saturday, August 21, 2021 11:23:12 AM (Elapsed Time: 2.27 seconds)

Saving Elevation Criteria...
Start Time: Saturday, August 21, 2021 11:23:09 AM
Succeeded at Saturday, August 21, 2021 11:23:12 AM (Elapsed Time: 2.27 seconds)

Saving Slope criteria...
Start Time: Saturday, August 21, 2021 11:23:09 AM
Succeeded at Saturday, August 21, 2021 11:23:12 AM (Elapsed Time: 2.27 seconds)

Saving Aspect criteria...
Start Time: Saturday, August 21, 2021 11:23:09 AM
Succeeded at Saturday, August 21, 2021 11:23:12 AM (Elapsed Time: 2.27 seconds)

Saving Final Raster (Addition)...
Start Time: Saturday, August 21, 2021 11:23:09 AM
Succeeded at Saturday, August 21, 2021 11:23:12 AM (Elapsed Time: 2.27 seconds)

Saving Final Raster (Multiply)...
Start Time: Saturday, August 21, 2021 11:23:09 AM
Succeeded at Saturday, August 21, 2021 11:23:12 AM (Elapsed Time: 2.27 seconds)

Spatial Analyst extension has been returned to the license manager.
```

31. Open Pro and add both final rasters



32. Open the attributes of each raster and compare your results





Answer the following questions:

1. What is the difference in using the add or multiply operator?
2. How do you interpret the results of each?